

PropCov: EFFECTIVE COVERAGE FOR PROPERTY-BASED TESTING

JESSE COULTAS, JOSEPH WISEMAN, AND LUÍS PINA

Accepted at
ISSTA 2026

PropCov
Artifact



<https://doi.org/10.5281/zenodo.17095525>

Measuring Property Quality

- PBT checks that a stated property holds for a SUT across many randomly generated, well-formed inputs.
- Coverage data from property test execution shows what methods and lines of code are reached.
- Traditional coverage tools report project-level coverage, not property-level impact.

Key Insight

Measure property coverage against
reachable code, not the whole project.

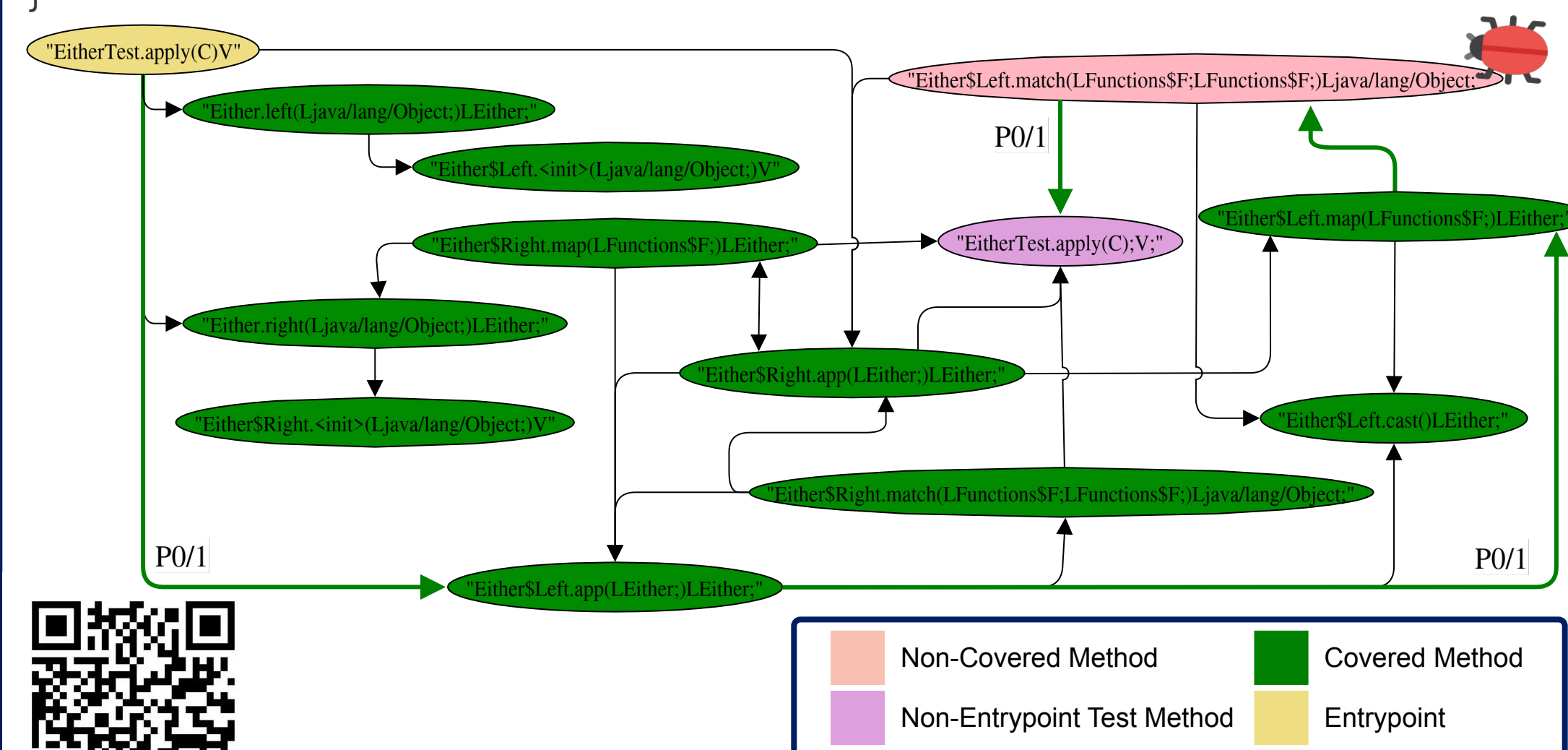
Motivating Example

Simple Property

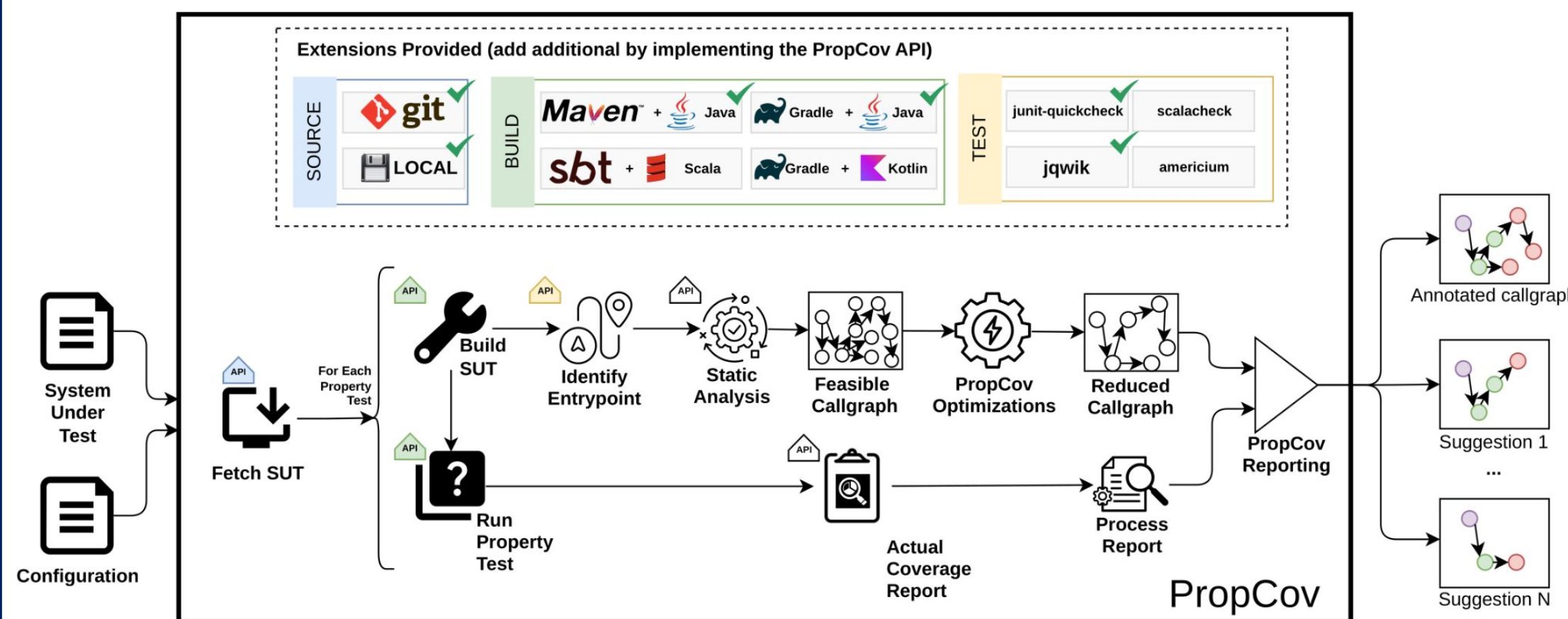
```
@Property
public void propertyTestAddOne(Integer x) {
    assertTrue(addOne(x) > x);
}
```

Complex Property

```
@Property
public void eitherApplyLeftShouldShortCircuit(char c, char d) {
    assertEquals(Either.right(String.valueOf(c)),
        Either.right(c).app(Either.right(Object::toString)));
    assertEquals(Either.left("fail"), Either.right(c).app(Either.left("fail")));
    assertEquals(Either.left(c), Either.left(c).app(Either.right(Object::toString)));
    //MISSING: Assume.assumeTrue(c != d);
    //MISSING: assertEquals(Either.left(c), Either.left(c).app(Either.left(d)));
}
```



PropCov Implementation



- PropCov analyzes Java property-based tests
- Static analysis approximates reachability per property
- Graph pruning removes external library code, paths that end in exceptions, and iterator interface calls
- Test execution collects actual coverage data
- Recommends per property coverage improvements

Finding: Coverage rarely improved after 100 trials

- Reran unchanged tests with increasing trial counts
- Classified coverage impact by trial threshold
- Only 8.4% improved beyond 100 trials

		Coverage per Test by Trial Count						Time (s) per Test by Trial Count					
cluster	n	1	10	100	500	1000		1	10	100	500	1000	
100% at 1	60	100%	100%	100%	100%	100%		12.34	12.43	12.70	13.89	25.97	
Max at 1	53	87%	87%	86%	87%	85%		13.67	13.77	14.19	15.12	25.02	
100% at 10	22	50%	100%	100%	100%	100%		10.17	10.18	10.43	10.91	14.91	
Max at 10	61	65%	89%	86%	86%	87%		16.14	16.22	17.20	24.09	56.68	
100% at 100	2	74%	95%	100%	100%	100%		15.64	15.16	16.48	15.98	16.50	
Max at 100	30	60%	75%	84%	78%	83%		10.89	11.00	11.52	17.83	60.08	
Max at 500	18	61%	80%	81%	88%	84%		10.97	11.05	11.75	15.34	32.60	
100% at 1000	1	92%	92%	92%	92%	100%		24.35	23.99	24.74	23.51	24.75	
Max at 1000	2	53%	68%	67%	73%	77%		8.65	8.59	8.70	8.83	23.42	

Why?

Trivial Tests: Simple or wrapper calls
Straight Line Code: No branching
Tests with Loops: Same instructions execute

JaCoCo vs PropCov

Project	PropCov			JaCoCo	
	Coverage	MR	UR (A+P)	Coverage	UR
3d-bin	101 (81%)	381 (20%)	0+3	482 (25%)	1454 (77%)
beam-zstd	5 (100%)	0 (0%)	0+0	5 (50%)	5 (50%)
browserstack	6 (86%)	0 (0%)	0+0	6 (1%)	433 (98%)
capsyl	1 (100%)	1 (1%)	0+0	2 (1%)	165 (99%)
convex	178 (15%)	121 (3%)	0+0	299 (6%)	3807 (79%)
dyn-shield	2 (5%)	3 (4%)	0+0	5 (7%)	24 (34%)
funcj	150 (74%)	41 (1%)	1+0	191 (5%)	4025 (96%)
hprofcrawler	28 (93%)	6 (2%)	0+0	34 (10%)	293 (90%)
im-collections	131 (89%)	56 (26%)	0+3	187 (86%)	80 (37%)
ipresource	28 (85%)	30 (11%)	0+0	58 (21%)	217 (78%)
JESA	7 (100%)	0 (0%)	0+0	7 (25%)	21 (75%)
jflex	67 (94%)	18 (3%)	0+0	85 (12%)	630 (89%)
jlsn	19 (79%)	13 (35%)	0+0	32 (86%)	8 (22%)
libosu	31 (86%)	6 (1%)	0+0	37 (9%)	395 (93%)
logish	21 (81%)	9 (2%)	0+0	30 (5%)	556 (94%)
manu	69 (97%)	30 (13%)	0+0	99 (44%)	156 (69%)
mph-table	34 (87%)	8 (2%)	0+0	42 (9%)	412 (91%)
nexappcore	0 (0%)	3 (3%)	0+1	3 (3%)	89 (97%)
occurrent	4 (100%)	0 (0%)	0+0	4 (6%)	60 (94%)
paillier4j	24 (100%)	6 (15%)	0+0	30 (73%)	13 (32%)
rdapd	1 (100%)	16 (2%)	0+0	17 (3%)	659 (97%)
rpki-commons	365 (96%)	45 (3%)	0+0	410 (27%)	1124 (74%)
sinch-sms	176 (99%)	85 (4%)	0+0	261 (11%)	2118 (92%)
stoa	190 (95%)	113 (24%)	8+0	303 (63%)	279 (58%)
wires	11 (55%)	7 (2%)	0+6	18 (4%)	383 (94%)
Average	66 (58%)	40 (5%)	9+13	106 (13%)	696 (85%)

Missed Reachable
Identified as not reachable
but is actually reachable

Unfeasible Reachable
identified as reachable
but is impossible to reach

PropCov Itself
Optimization included
impossible node

Underlying Analysis
Analysis (RTA) found
impossible node

Improved Coverage

Project	# Props	Methods				LOCs			
		Total	Before	After	Improv	Total	Before	After	Improv
funcj	1	16	15	16	100%	17	16	17	100%
jflex	7	46	35	46	100%	363	193	297	61.18%
manu	3	40	32	38	75%	261	184	210	33.77%
mph-table	1	27	21	24	50%	120	52	83	45.59%
sinch-sms	4	102	98	102	100%	297	272	284	48%
stoa	10	245	178	193	22.39%	1085	696	776	20.57%
wires	16	20	13	17	57.14%	43	29	37	57.14%
TOTAL	42	Average: 72.08%				Average: 52.32%			

- 72% coverage increase on unfamiliar SUT projects
- Ranked targets made coverage gaps actionable
- Focused generator/test changes reached missed code

